

LABORATORIO DI SISTEMI OPERATIVI

Posix Threads: creazione, join e detach

fabio.rossi@unipg.it

075 5855005

Posix Threads

- parti di uno stesso programma eseguite contemporaneamente e schedate indipendentemente l'una dall'altra
- in altre parole: "*più tracce di esecuzione concorrenti in un singolo processo*" (slide Prof. Carpi)
- un singolo processo può "contenere" più threads
- condividono la memoria globale (*data and heap*) ma ognuno ha il proprio *stack*

Posix Threads

- *POSIX.1c, Threads extensions ([IEEE Std 1003.1c-1995](#))*
- NON è un'implementazione, è uno standard
- Implementati in molti SO... ma con delle differenze
- definisce una serie di interfacce (funzioni, file *header*) per la programmazione di threads
- conosciuti come *POSIX Threads* o *Pthreads*

Posix Threads: implementazioni in *LINUX*

- ***man 7 pthreads***
- nel corso del tempo due implementazioni nella *GNU C library*:
 - **LinuxThreads**

Implementazione originale. Non più supportata (a partire da *glibc 2.4*)
 - **NPTL (Native POSIX Threads Library)**
 - implementazione "più moderna"
 - più rispondente allo standard *POSIX.1*
 - prestazioni migliori se si crea un gran numero di threads
 - disponibile a partire da *glibc 2.3.2* e richiede caratteristiche presenti a partire da Linux 2.6
- **di seguito faremo riferimento a *NPTL* in *Linux***

Posix Threads: implementazione in LINUX

- Per determinare la versione di glibc:

ldd --version

- Per conoscere quale implementazione dei threads:

getconf GNU_LIBPTHREAD_VERSION

POSIX.1: attributi condivisi (*process-wide*) (1)

POSIX.1 richiede che i seguenti attributi siano condivisi tra i threads di uno stesso processo:

- process ID
- parent process ID
- process group ID and session ID
- controlling terminal
- user and group IDs
- open file descriptors
- record locks (see *fcntl(2)*)
- signal dispositions

POSIX.1: attributi condivisi (*process-wide*) (2)

- file mode creation mask (*umask(2)*)
- current directory (*chdir(2)*) and root directory (*chroot(2)*)
- interval timers (*setitimer(2)*) and POSIX timers (*timer_create(2)*)
- nice value (*setpriority(2)*)
- resource limits (*setrlimit(2)*)
- measurements of the consumption of CPU time (*times(2)*) and resources (*getrusage(2)*)

POSIX.1: attributi NON condivisi (*per-thread*)

- thread ID (the *pthread_t* data type)
- signal mask (*pthread_sigmask(3)*)
- the ***errno*** variable
- alternate signal stack (*sigaltstack(2)*)
- real-time scheduling policy and priority (*sched(7)*)

Posix Threads: ritorno delle funzioni

- La maggior parte delle funzioni *pthread* **tornano 0** se non ci sono errori ed un codice di errore altrimenti
- questo codice di errore ha lo stesso significato dei valori convenzionali di ***errno***
- ... ma le funzioni Pthreads **NON settano *errno***
- *POSIX.1-2001* stabilisce che *EINTR* non può essere ritornato

Posix Threads: Threads IDs

- Ogni thread ha un unico *thread ID** utilizzabile nelle funzioni *pthread*s
- è tornato al chiamante da *pthread_create(3)*
- ... o può essere ottenuto con *pthread_self(3)*
- sono unici nel processo ma non nel sistema
- il SO riusa gli *ID* di thread terminati
- se un'applicazione usa un ID di un thread terminato il risultato non è definito

(*): da non confondere con il *Thread ID* del sistema (*ps --help threads* per le opzioni di *ps*) visibile con *ps* ed ottenibile con *gettid(2)* o *syscall(SYS_gettid)*

Posix Threads: Thread Safety

- **POSIX Safety Concepts:** definiscono se una funzione può essere chiamata in determinati contesti senza che si verifichino comportamenti indesiderati
- ***MT-Safe*** (*MT* sta per *Multi-threading*) o ***Thread-Safe***: una funzione lo è se può essere chiamata contemporaneamente da più thread senza modificare il comportamento
- *MT-Safe* non significa che l'esecuzione sia atomica, né che usi alcun meccanismo di sincronizzazione nell'accesso alla memoria
- è possibile che chiamare più funzioni *MT-Safe* in sequenza possa sortire effetti diversi da un'esecuzione atomica delle stesse; chiamate concorrenti da altri *thread* possono interferire
- vedi ***man 7 attributes*** per chiarimenti su *MT-Safe*

Posix Threads: compilazione

- includere ***pthread.h***
- se richiesto compilare col flag ***-pthread***

pthread_create(3): creazione di threads (1)

```
#include <pthread.h>

int pthread_create(pthread_t *thread,
                  const pthread_attr_t *attr,
                  void *(*start_routine) (void *),
                  void *arg);
```

- avvia un nuovo thread nel processo chiamante
- imposta in **thread** il *thread ID* del *thread* creato che può essere utilizzato dalle altre funzioni *pthread*
- il thread avvia l'esecuzione chiamando la funzione **start_routine**
- **arg** viene passato come unico argomento a **start_routine**
- un esempio d'uso in *man 3 pthread_create*

pthread_create(3): creazione di threads (3)

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *thread,  
                  const pthread_attr_t *attr,  
                  void *(*start_routine) (void *),  
                  void *arg);
```

- **attr** punta ad una struttura di tipo *pthread_attr_t* che può essere usata per settare alcuni attributi del nuovo thread, per esempio per crearlo *detached* (vedi oltre)
- **attr** può essere inizializzata con *pthread_attr_init(3)*.
- Se è *NULL* sono usati gli attributi di default
- *pthread_create(3)* torna **0** per successo; un codice di errore altrimenti

Pthreads: terminazione di un thread

- un thread può terminare:
 - con `pthread_exit(3)` specificando un codice d'uscita disponibile per un altro thread del processo che abbia chiamato `pthread_join(3)`
 - quando **start_routine()** (vedi `pthread_create()`) termina con **return**. Equivalente a chiamare `pthread_exit(3)`
 - è cancellato con `pthread_cancel(3)`
 - un *thread* chiama `exit(3)` o quello principale chiama `return`. L'intero processo termina (e tutti i suoi threads)!

Pthreads: terminazione di un thread

```
int pthread_exit(void *retval) ;
```

- termina il thread chiamante
- se il thread è *joinable* (vedi più avanti), il valore ritornato in **retval** è reso disponibile ad altro thread del processo che chiami *pthread_join(3)*
- il valore cui **retval** punta non dovrebbe essere allocato nello stack del chiamante, perché dopo la terminazione del thread il contenuto dello stack è "*undefined*"

Pthreads: Primo esempio. *mt1.c*

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
int iCondivisa = 13;
void thread1(int *arg) {
    int iLocale = 101;
    printf("Sono il primo thread. Parametro = %d, var. condivisa = %d, locale = %d\n", *arg, iCondivisa, iLocale);
    pthread_exit(NULL); }
void thread2(char *arg) {
    int iLocale = 202;
    printf("Sono il secondo thread. Parametro = \"%s\", var. condivisa = %d, locale = %d\n", arg, iCondivisa, iLocale);
    pthread_exit(NULL); }
void main()
{
    pthread_t th1, th2; int arg1 = 1; char arg2[] = "Ciao";
    pthread_create(&th1, NULL, (void *)thread1, (void *)&arg1);
    pthread_create(&th2, NULL, (void *)thread2, (void *)arg2);
    sleep(2);
    return;
}
```

(adattamento da slide del *Prof. Mumolo - Univ. Trieste*)

Pthreads: qualche osservazione su mt1.c

- variabili globali condivise tra threads
- comunicazione tra threads realizzabile con le risorse (variabili) condivise ma non c'è nessuna sincronizzazione
- *stack* (var. locali) specifico per ogni thread
- nessun vincolo alla schedulazione dei threads. *POSIX* definisce funzioni che consentono di modificare schedulazione e priorità
- ma... a cosa serve *sleep()*?! Provate a toglierla...

Pthreads: join e detach

- un thread può essere *joinable* o *detached*
- di default i threads sono creati *joinable*
- se un thread è *joinable* un altro thread può richiamare *pthread_join(3)* per attendere che termini e per ottenere il suo *exit status*

Pthreads: threads joinable

- le risorse allocate per un thread *joinable* sono rilasciate solo se un altro thread chiama *pthread_join(3)* per attenderlo
- Se nessun *join* viene fatto il thread terminato è in stato **zombie** e le risorse sono rilasciate solo all'uscita dal processo. Se i threads *zombie* si accumulano oltre misura non sarà più possibile creare altri thread

Pthreads: threads joinable

- ogni thread nel processo può attendere con *pthread_join(3)* qualsiasi altro thread (del processo stesso)
- Non c'è per i threads un'operazione analoga a quella vista per i processi *child* (*fork(2)*):
waitpid(-1, &status, 0)
cioè una chiamata che esegue *join* su qualsiasi thread termini

Pthreads: thread detached

- E' possibile creare un thread *detached* modificandone gli attributi in fase di creazione con *pthread_attr_setdetachstate(3)* oppure richiamando *pthread_detach(3)* per un thread già creato
- le risorse allocate per un thread *detached* sono subito liberate alla sua terminazione
- non è possibile eseguire il *join* ad un thread *detached* per ottenere il suo *exit status*, né è possibile renderlo *joinable*
- creare thread *detached* può essere utile quando non ci curiamo dell'*exit status* dei thread; per esempio nello sviluppo di certi *daemon*

Pthreads: conclusioni su join e detach

- quindi, una tra *pthread_join()* e *pthread_detach()** dovrebbe essere chiamata per ogni thread creato affinché le risorse vengano liberate
- fermo restando che, all'uscita del processo, tutte le risorse dei thread sono liberate comunque

(*): se non è stato creato già *detached* settandone gli attributi

Pthreads: joining threads

```
int pthread_join(pthread_t thread,  
                 void **retval);
```

- Attende che il thread specificato da **thread** termini (se è *joinable*)
- se **retval** non è *NULL* riceve il valore tornato da *pthread_exit(3)* oppure *PTHREAD_CANCELED* se il thread è stato cancellato
- se più thread cercano simultaneamente di fare il *join* allo stesso thread il risultato non è definito (dallo standard)

Pthreads: detaching threads

```
int pthread_detach(pthread_t thread) ;
```

- rende il thread specificato da **thread** *detached*

Pthreads: *mt1.c* con join

```
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
int iCondivisa = 13;
void thread1(int *arg) {
    int iLocale = 101;
    printf("Sono il primo thread. Parametro = %d, var. condivisa = %d, locale = %d\n", *arg, iCondivisa, iLocale);
    pthread_exit((void *)1000); }
void thread2(char *arg) {
    int iLocale = 202;
    printf("Sono il secondo thread. Parametro = \"%s\", var. condivisa = %d, locale = %d\n", arg, iCondivisa, iLocale);
    pthread_exit((void *)2000); }
void main()
{
    pthread_t th1, th2; int arg1 = 1; char arg2[] = "Ciao"; long l1, l2;
    pthread_create(&th1, NULL, (void *)thread1, (void *)&arg1);
    pthread_create(&th2, NULL, (void *)thread2, (void *)&arg2);
    pthread_join(th1, (void **)&l1);
    pthread_join(th2, (void **)&l2);
    printf("\nRitorno thread 1 = %ld\nRitorno thread 2 = %ld\n", l1, l2);
    return;
}
```

(adattamento da slide del Prof. Mumolo - Univ. Trieste)